

PONTIFÍCIA UNIVERSIDADE CATÓLICA DO RIO GRANDE DO SUL (PUCRS)

Escola Politécnica | Sistemas Operacionais – 2026/II | Prof. Filipo Mór

Contagem Paralela de Objetos em uma Matriz Binária

Implementações Sequencial e Paralela com Pthreads e Union-Find (DSU) em ANSI C
(C89/C90)

Apresentação do Trabalho Prático 1 (Tempo: 10 minutos)

Repositório Público: github.com/filipepereira03/T1-SISOP

1. O Desafio e a Conectividade 8

Definição do problema e desafio da divisão concorrente de dados espaciais

O que é um Objeto?

- **Matriz Binária:** Valores 0 representam o fundo e 1 representam células de primeiro plano.
- **Conectividade 8 (8-Vizinhança):** Duas células de valor 1 pertencem ao mesmo objeto se tocam por **arestas** (horizontal/vertical) OU por **cantos** (diagonais).
- **Componente Conexo Maximal:** Cada objeto é um agrupamento fechado onde todos os pontos com 1 se alcançam mutuamente.

O Desafio Central da Paralelização

- **Corte nas Fronteiras:** Um objeto de grande escala pode atravessar várias faixas atribuídas a threads diferentes.
- **Falha da Soma Simples:** Apenas somar a quantidade de objetos encontrados por cada thread gera **contagens duplicadas** e resultados inconsistentes.
- **Objetivo:** Descobrir componentes locais de forma concorrente e **unificar deterministicamente** as regiões que se tocam nas bordas.

2. Implementação Sequencial de Referência

Flood fill iterativo com pilha explícita na memória dinâmica

Eliminação de Recursão Profunda

- **Risco Recursivo:** Em matrizes de 1.000.000 de células, um objeto longo pode disparar recursão de milhares de níveis, estourando a pilha de execução (*Stack Overflow*).
- **Solução:** Pilha dinâmica explícita (Stack) alocada na **heap** com redimensionamento sob demanda.
- **Complexidade:** $O(N)$ em tempo e $O(N)$ em memória para N células.

Mecanismo de Visitação e Ground Truth

- **Marcação no Empilhamento:** Cada célula é marcada como visitada no momento exato em que é adicionada à pilha, impedindo que múltiplos vizinhos a empilhem novamente.
- **Padrão Ouro de Correção:** A versão sequencial processa 100% da matriz sem interferências e serve como base comparativa obrigatória para a versão paralela.
- **Padrão ANSI C (C89/C90):** Zero dependências de C99/C11, compatível com Linux e macOS.

3. Decomposição de Domínio (Row-Striping)

Divisão por faixas de linhas, localidade de cache e rotulagem local

📌 Como as Linhas são Fatiadas?

- A matriz de R linhas é dividida entre T threads em blocos contíguos de linhas `[start_row, end_row)`.

- **Distribuição Justa do Resto:**

```
base_rows = rows / num_threads; remainder = rows %  
num_threads; count = base_rows + (thread_id < remainder ? 1 :  
0);
```

- **Localidade Espacial de Cache:** Em C, matrizes são *Row-Major*. Linhas contíguas garantem acessos sequenciais rápidos na memória cache L1/L2.

🏷️ Fase 1: Flood Fill Concorrente

- Cada thread executa seu próprio flood fill restrito à sua faixa.
- **ID Único sem Locks:** Cada novo componente local recebe um ID único baseado na posição da sua célula inicial:

```
comp_id = r * cols + c + 1; /* Estritamente positivo */
```

- Zero comunicação e zero contenção de memória durante a Fase 1! As threads trabalham com paralelismo máximo.

4. Consolidação de Fronteiras com Union-Find (DSU)

Barreira de sincronização portátil, conectividade 8 e resolução transitiva

👉 Barreira Portável e Inspeção de Bordas

- **Compatibilidade macOS/Linux:** Implementação de barreira com `pthread_mutex_t` e `pthread_cond_t` (pois `pthread_barrier_t` não existe no macOS).
- **Checagem Concorrente de Fronteira:** Cada thread verifica sua última linha (`r_top`) contra a primeira linha da faixa seguinte (`r_bot`):
 - Diagonal esquerda: (`r_bot`, `c - 1`)
 - Vertical abaixo: (`r_bot`, `c`)
 - Diagonal direita: (`r_bot`, `c + 1`)

🌲 Por que Union-Find (DSU) com Mutex?

- Ao detectar conexão entre rótulos `label_top` e `label_bot`, executa:

```
pthread_mutex_lock(&dsu_mutex); dsu_union(&dsu, label_top,  
label_bot); pthread_mutex_unlock(&dsu_mutex);
```

- **Transitividade Automática:** Objetos em forma de "U" ou serpentes que cruzam 3 ou 4 faixas são fundidos na mesma raiz canônica.
- **Contagem Global Final:** Itera sobre as raízes canônicas com `dsu_find`, contabilizando cada objeto exatamente 1 vez!

5. Tabela 7.1 - Validação das Matrizes Obrigatórias

Resultados 100% exatos com as 5 matrizes oficiais do enunciado da PUCRS

EXEMPLO	DIMENSÕES	OBJETOS ESPERADOS	SEQUENCIAL	PARALELO (4 THREADS)	STATUS	CARACTERÍSTICAS TESTADAS
Exemplo 1	5 × 5	3	3	3	OK	Casos compactos e referência básica
Exemplo 2	6 × 8	4	4	4	OK	Objeto atravessando fronteiras
Exemplo 3	8 × 8	5	5	5	OK	Encontro de 4 blocos e conexões diagonais
Exemplo 4	9 × 12	6	6	6	OK	Objetos irregulares ocupando múltiplos blocos
Exemplo 5	12 × 12	7	7	7	OK	Travessia diagonal longa ao longo de 3 faixas

Testes também disponíveis pelo terminal com --test, usando 1, 2 ou 4 threads.

6. Desempenho e o Enigma do Ponto 38

Medições repetidas (10 rodadas) e explicação de quando o paralelo é mais lento

Matriz 1000×1000: resultados registrados

CONFIGURAÇÃO	TEMPO MÉDIO	SPEEDUP (S)
Sequencial	0.0303 s (±0.008s)	1.00x
Paralelo 2T	0.0231 s (±0.003s)	1.31x
Paralelo 4T	0.0211 s (±0.008s)	1.44x
Paralelo 8T	0.0269 s (±0.007s)	1.13x

Médias de 10 rodadas no ambiente original. Tempos e matriz gerada por rand() podem variar em outro sistema.

? Requisito 38: Por que o Paralelo perde em matrizes pequenas?

- **Matriz 5×5:** Sequencial levou 1.0 μs enquanto o Paralelo levou 1705 μs (~1700x mais lento!).
- **Custo de Criação de Threads:** Chamar pthread_create e pthread_join envolve o kernel do SO e alocação de pilhas, consumindo dezenas de microssegundos.
- **Granularidade Insuficiente:** Processar 25 células é tão rápido que a sobrecarga (*overhead*) de coordenação supera em muito a computação útil.

7. Conclusões e Ferramentas Desenvolvidas

Síntese dos resultados, domínio do trabalho e diferenciais do projeto

Objetivos Alcançados

- **Conformidade Rigorosa:** Padrão ANSI C (C89/C90) estrito com compilação limpa em `gcc -std=c89 -Wall -Wextra -pedantic -pthread`.
- **Sincronização Segura:** Barreira customizada para máxima portabilidade (macOS + Linux) e exclusão mútua garantida nas uniões de DSU.
- **Estrutura Completa do Repositório:** Código-fonte, testes automatizados, resultados de benchmarks e slides em PDF.

Demonstração e Visualizador Web

- **Modo Terminal:** Flags `--visual` para ver os objetos coloridos no terminal e demarcação de faixas de threads.
- **Visualizador Interativo:** Aplicativo web embutido (`visualizador.html`) permitindo desenho com o mouse, inspeção de fronteiras DSU e testes ao vivo.
- **Repositório Público:**
github.com/filipepereira03/T1-SISOP